

Multi-Dimensional Generalizations of Binary Search Trees

Design and Analysis of Algorithms

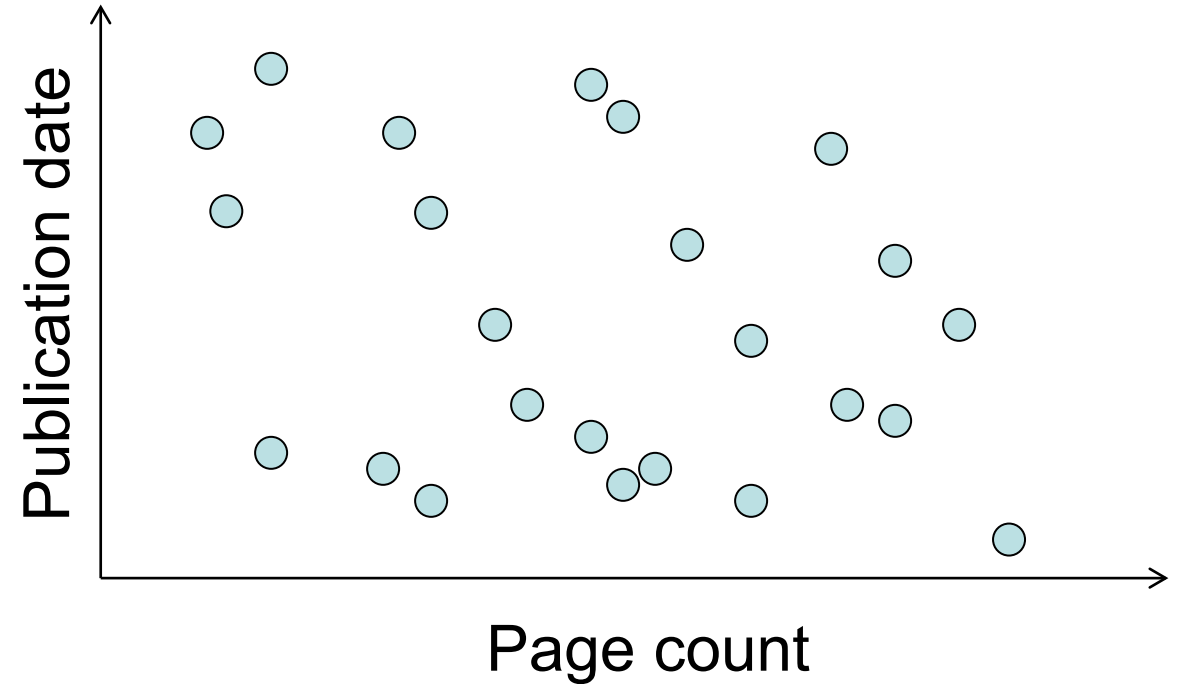
Brian C. Dean

**School of Computing
Clemson University**



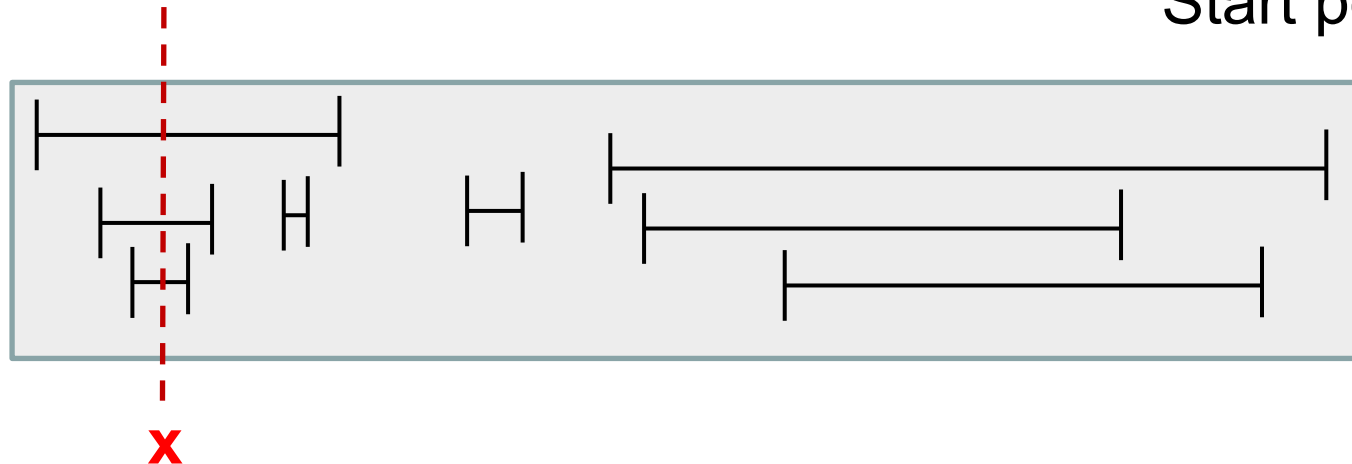
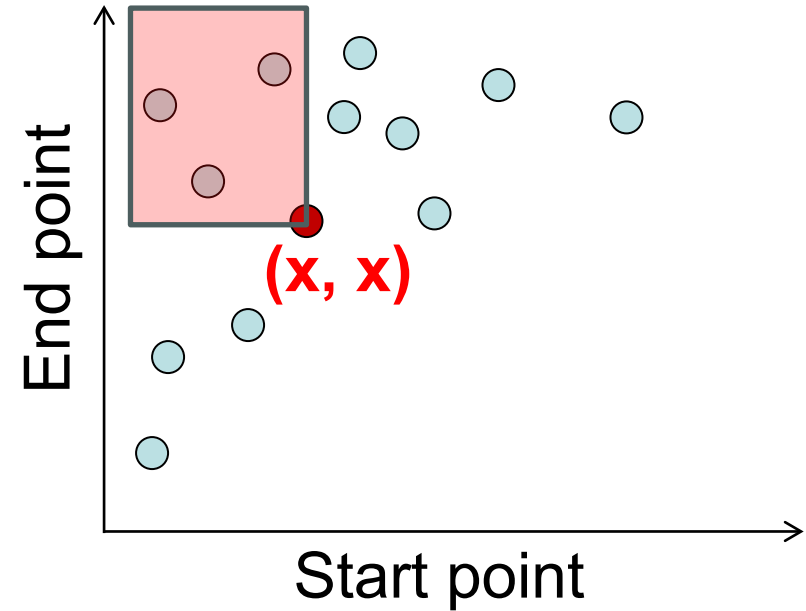
Multi-Dimensional Data is Everywhere...

- A database can often be regarded as N points in D dimensions (i.e., N records each with D fields).
- AI often deals with points in a high-dimensional feature space.
- As often is the case, tools that work in low dimensions often don't scale well to high-dimensional applications (which can often be quite challenging!)

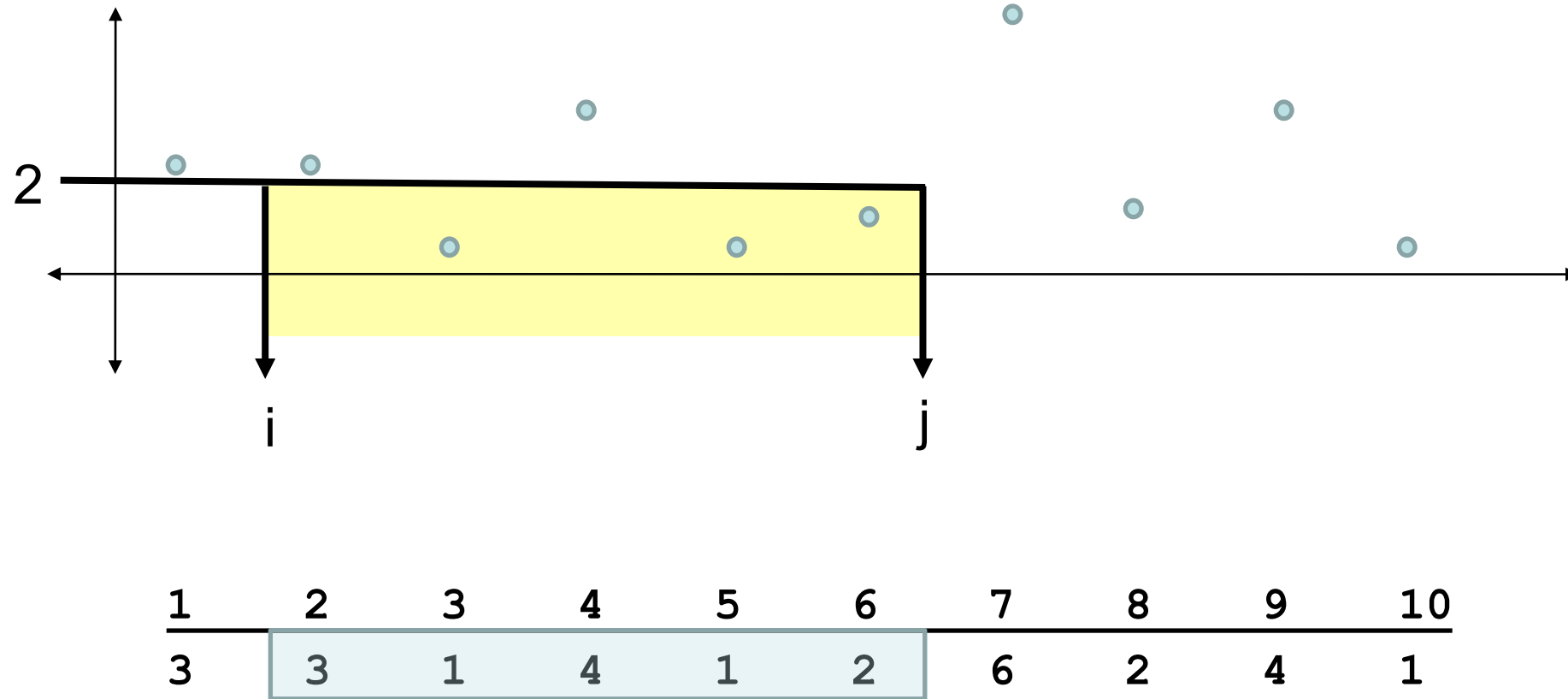


Multi-Dimensional Data is Everywhere...

- Many problems that may not initially look “high dimensional” can often be regarded as such.
- E.g., “Stabbing” queries in a set of N intervals on the number line.



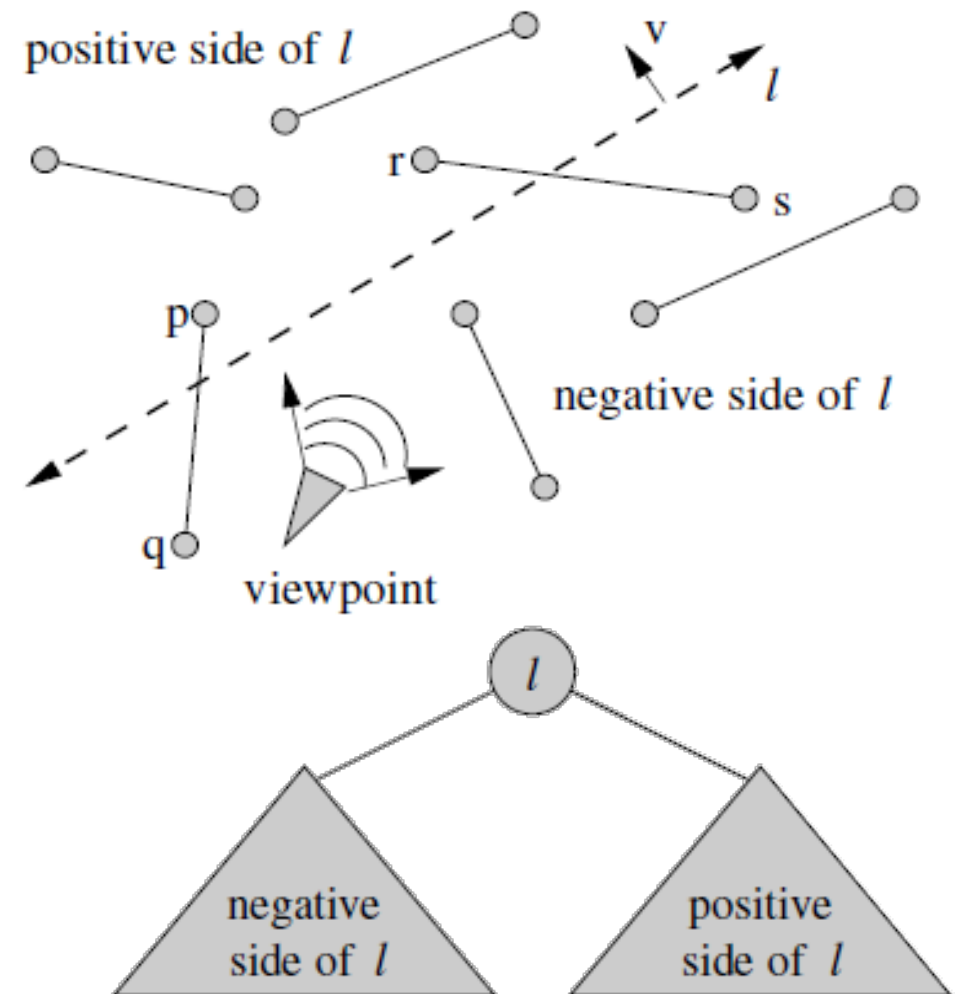
Multi-Dimensional Data is Everywhere...



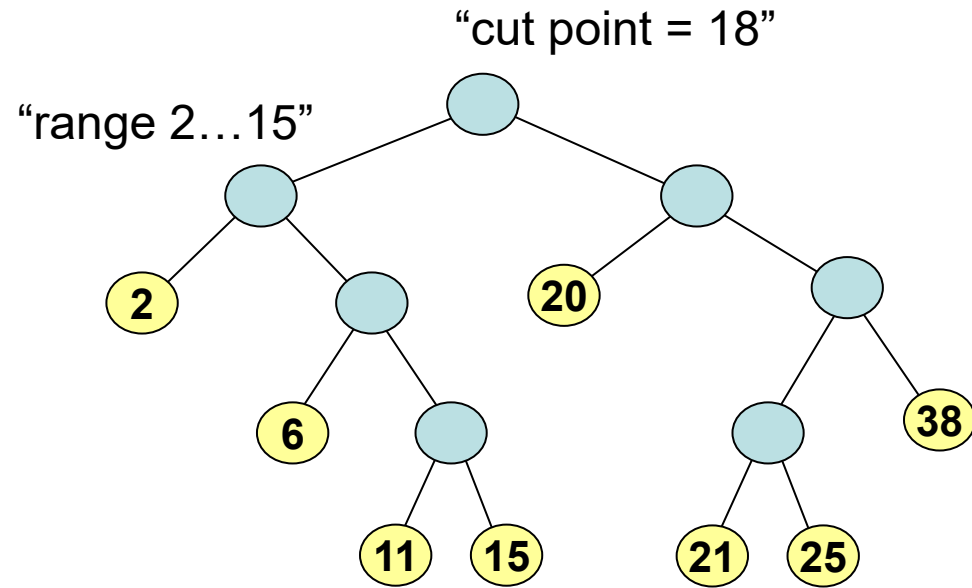
“Tell me all the values at most 2 in a subarray $A[i...j]$ ”

Binary Space Partition Trees

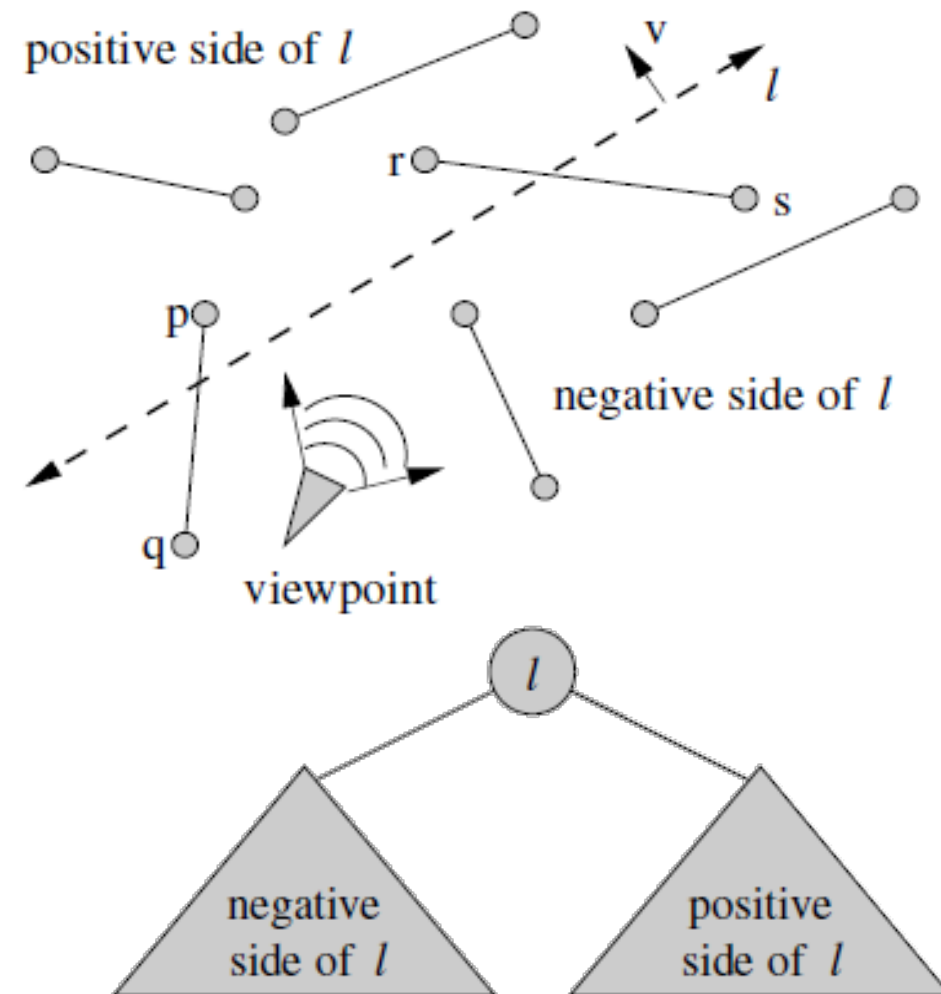
- Natural multi-dimensional way to generalize the binary search tree.
- Root: line in 2D, plane in 3D, or hyperplane in higher dimensions.
- If storing objects other than points, they get split by the root.
- Nice application in 3D rendering: traverse tree to apply the “painter’s algorithm”, rendering scene contents from back to front from any viewpoint.



Aside: Leaf-Oriented Trees

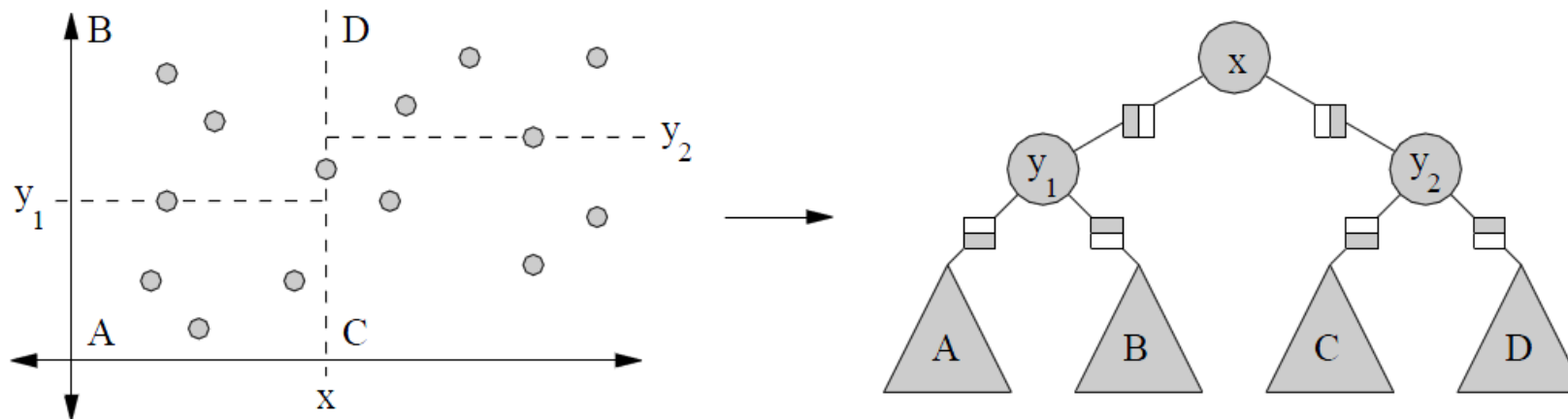


Note: # interior nodes < # leaves
(as long as all interior nodes branching)



Interleaving Dimensions: The kd-Tree

- A binary space partition tree with alternating cuts parallel to the coordinate axes.
- First split (at root) in the x direction, next level on y, and so on.
- In $d > 2$ dimensions, we cycle through splits along each dimension as we move down the tree.
- $O(n \log n)$ build time, $O(n)$ space, $O(\log n)$ height.

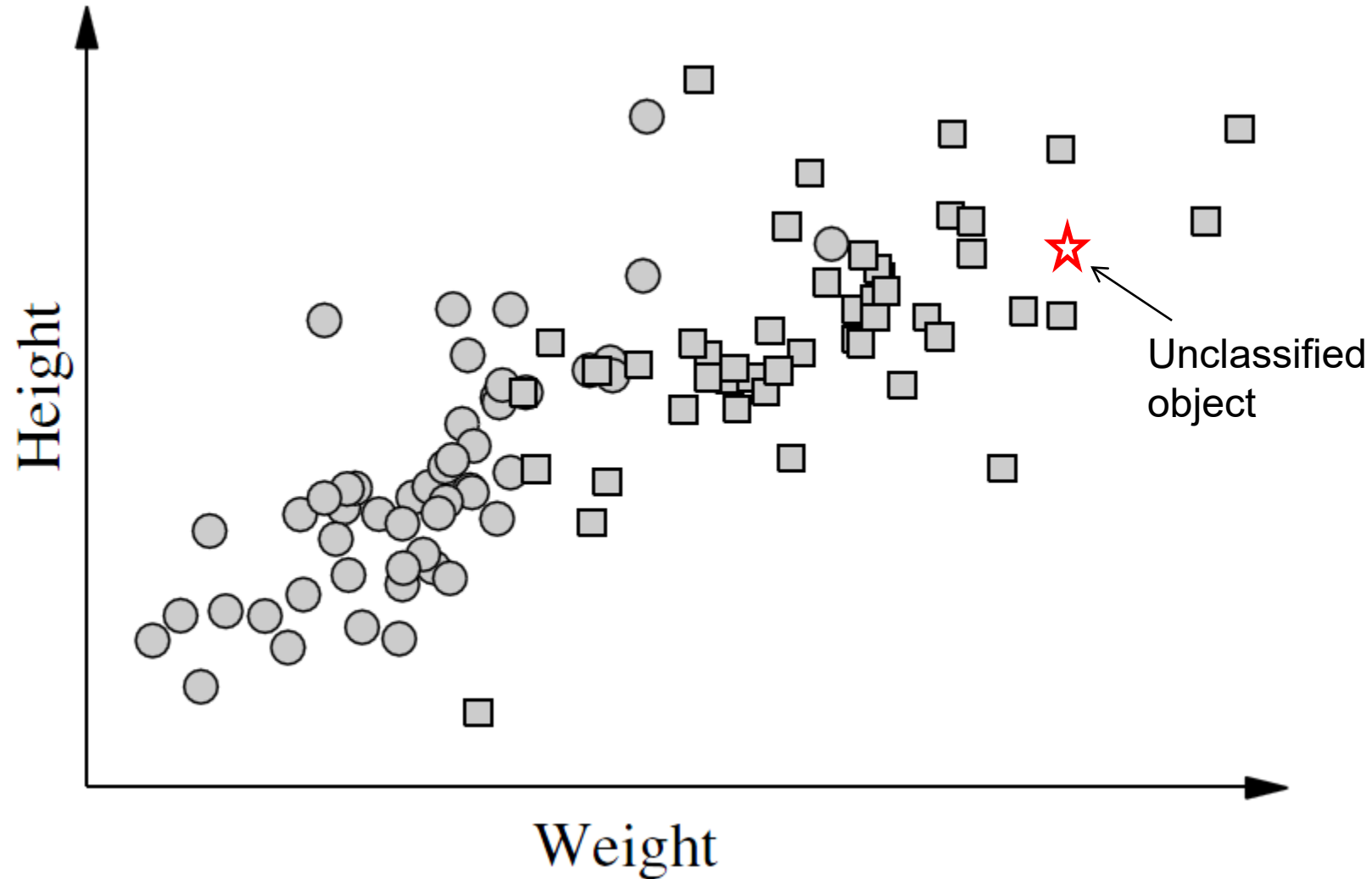


Applications of kd-Trees: Fast Ray Tracing!



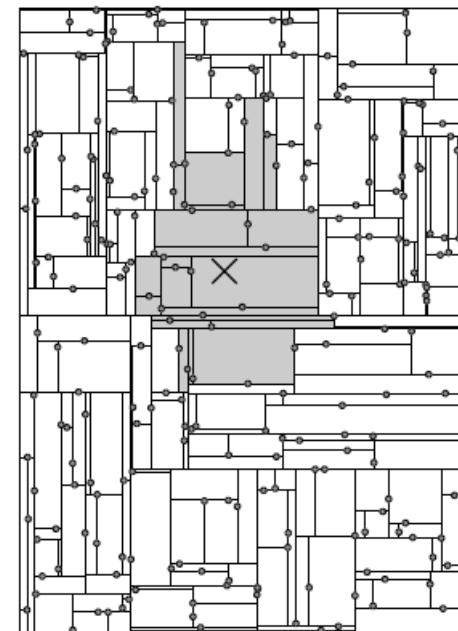
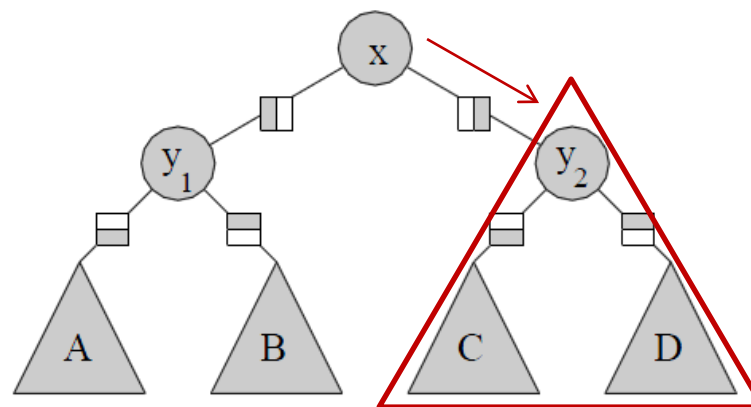
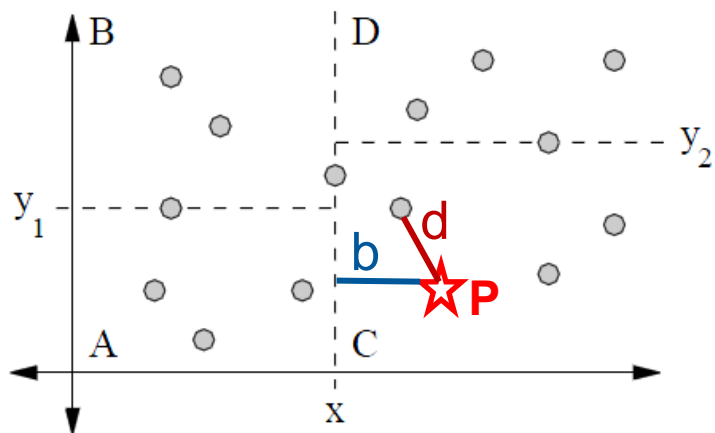
Applications of kd-Trees:

(k-) Nearest-Neighbor Classification



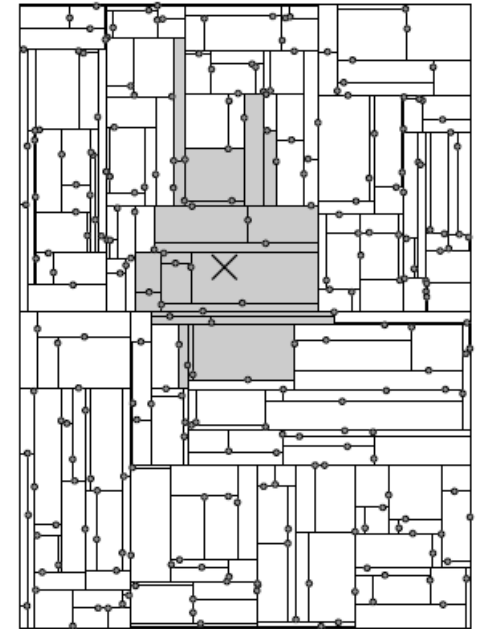
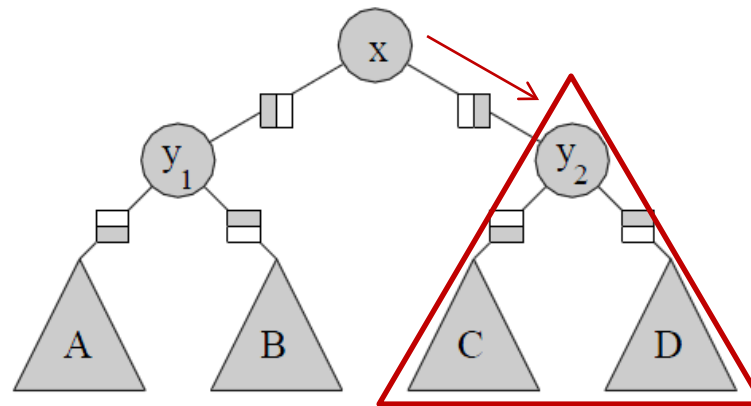
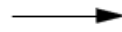
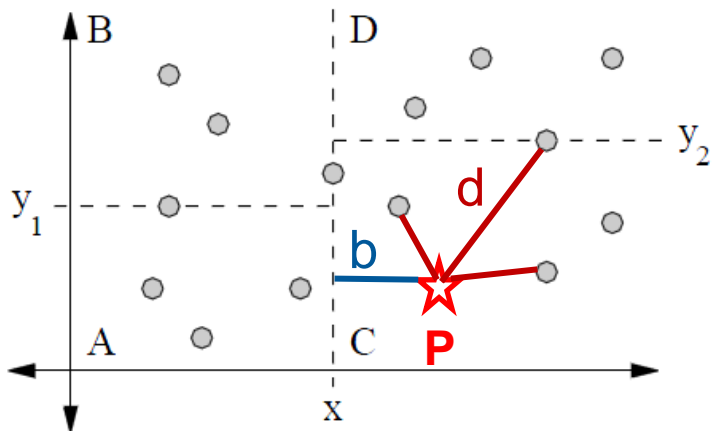
Nearest Neighbor Search

- Use a “pruned” traversal guided by target point p .
- Keep global variable with nearest neighbor so far.
- When visiting root:
 - Check if point stored at root is new nearest neighbor.
 - Recursively search subtrees, starting in the direction of the target point p .
 - Let d be nearest neighbor distance we found so far.
 - Determine if it's worth searching the other subtree...
(no, if $d < b$, where b = dist to split in current coordinate)



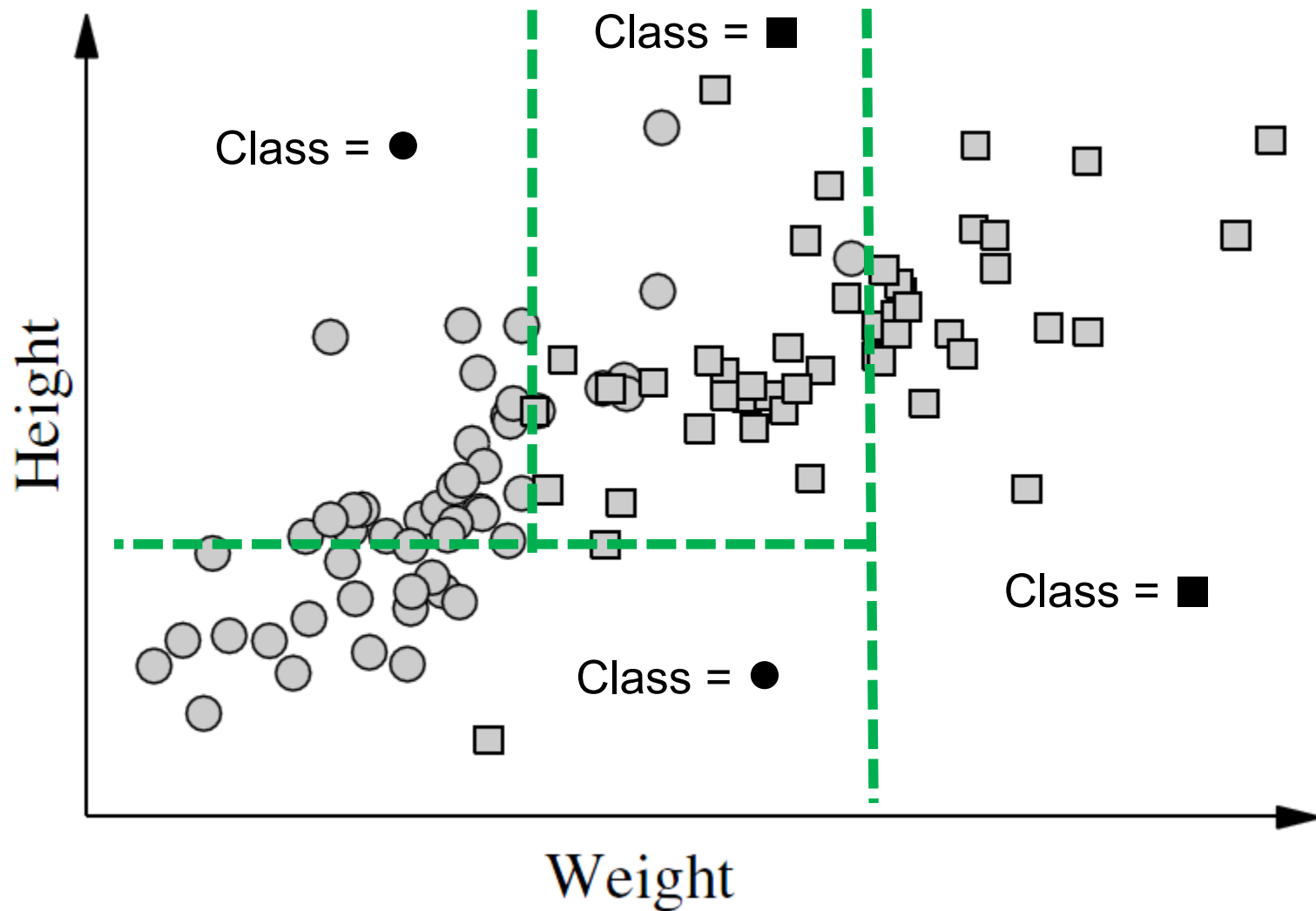
k-Nearest Neighbor Search

- Keep (max-) priority queue of k nearest neighbors.
- When visiting root:
 - Check if point stored at root belongs in current set of k nearest neighbors.
 - Recursively search subtrees, starting in the direction of the target point p.
 - Let d be the farthest distance among the k nearest neighbors found so far.
 - Determine if it's worth searching the other subtree...
(no, if $d < b$, where b = dist to split in current coordinate)



Classification with Decision Trees

Does this Picture Look Familiar?



The Quadtree

- Root node splits into 4 quadrants at some point (usually $(x_{\text{mid}}, y_{\text{mid}})$, although random is usually fine).
- Like kd-tree, subdivide until ≤ 1 point in region.
- $O(n \log n)$ build time, $O(n)$ space, $O(\log n)$ height.
- Generalizes naturally to $d = 3$ (octrees) and higher dimensions, but scales very poorly with dimension...

